



SISTEMAS BIOINSPIRADOS

*MAGÍSTER EN INTELIGENCIA
ARTIFICIAL*

AGOSTO 2026 - SEPTIEMBRE 2026

PROFESOR: RICARDO CONTRERAS A.

RE-PRESENTACIÓN

- Ricardo Contreras A. (M.Sc. Computer Science)
- Áreas de desarrollo: Vida Artificial, Teoría de Computación
- E-mail: rcontrer@udec.cl, ricardo.contreras@edu.uai.cl

TÓPICOS DEL CURSO

- Algoritmos evolutivos
 - Algoritmos genéticos simples
 - Algoritmos genéticos avanzados
- Swarm intelligence
 - Colonias de hormigas (Ant colony)
 - Abejas artificiales
- Sistemas dinámicos
 - Autómatas celulares
- Nuevas miradas evolutivas
 - Bacterias

BIBLIOGRAFÍA BÁSICA

- Goldberg, David
 - Genetic algorithms in search, optimization and machine learning
 - Reading, Mass. Addison Wesley Pub., 1989
- Górriz, J.M., Contreras, R., Pinninghoff, M.A. et al.
 - Computational Approaches to Explainable Artificial Intelligence: Advances in Theory, Application and Trends
 - Information Fusion, Elsevier, December 2023
- Michalewicz, Zbigniew
 - Genetic algorithms + data structures = evolution programs
 - Springer, Berlin, 1992
- Mitchell, Melanie
 - An introduction to genetic algorithms
 - Cambridge, MIT Press, 1996

INTRODUCCIÓN

- Algoritmos que operan en analogía con procesos naturales
- Clave: Principio de evolución (supervivencia de los más “adecuados”)
- La existencia de computadores masivamente paralelos hace que estas técnicas tengan un interés práctico

INTRODUCCIÓN

- Después que los científicos se desilusionaron con los intentos clásicos de modelamiento de la inteligencia, empezaron a mirar en otras direcciones.
- Dos campos relevantes surgieron; conexionismo (redes neuronales, procesamiento paralelo) y computación evolutiva.
- Es el segundo campo el que nos interesa hoy: algoritmos genéticos (también cabe acá la programación genética).

¿QUÉ SON LOS ALGORITMOS GENÉTICOS?

- Un **algoritmo genético** (o **AG**) es una técnica de búsqueda usada en computación para encontrar soluciones óptimas o aproximadas a problemas de búsqueda y de optimización.
- Los algoritmos genéticos encajan en la categoría de heurísticas de búsqueda global.
- Son una clase de algoritmos evolutivos inspirados en la evolución biológica, consideran la herencia, la mutación, la selección y el cruzamiento (recombinación).

¡¡¡¡ESTOS ALGORITMOS GENÉTICOS!!!!

- Se implementan como una simulación computacional en la cual una población de representaciones abstractas (llamadas cromosomas) de candidatos a ser soluciones (individuos) a un problema, evoluciona hacia soluciones mejores.
- Tradicionalmente, las soluciones se representan como strings de 0s y 1s (representación binaria), pero se pueden usar otras codificaciones.

¡¡¡¡ ESTOS ALGORITMOS GENÉTICOS!!!!

- La evolución generalmente se inicia desde una población de individuos generados en forma aleatoria, y a cada paso evolutivo se le denomina generación.
- En cada generación, se evalúa la aptitud de cada individuo (el fitness) en la población.
- Se seleccionan múltiples individuos de la población (basado en su fitness), y son modificados (recombinados y posiblemente mutados), para formar una nueva población.

!!!! ESTOS ALGORITMOS GENÉTICOS!!!!

- La nueva población es usada en la próxima iteración del algoritmo.
- Comunmente, el algoritmo termina cuando se llega a un máximo de generaciones o se ha alcanzado una respuesta satisfactoria en términos de la población.
- Si el algoritmo termina debido a que se ha alcanzado un número máximo de generaciones, la solución podría no ser satisfactoria.

TERMINOLOGÍA

- **Individual** – Una solución posible
- **Población** - Grupo que contiene a todos los *individuos*
- **Espacio de búsqueda** - Todas las posibles soluciones al problema
- **Cromosoma** – La representación de un *individuo*
- **Rasgo** – Aspecto posible (*características*) de un *individuo*
- **Alelo** – Conjunto posible de rasgos (moreno, rubio, etc.)
- **Locus** - La posición de un gen en un *cromosoma*
- **Genoma** - Colección de todos los cromosomas de un *individuo*

EN LOS ALGORITMOS GENÉTICOS...

- Una población de individuos (soluciones potenciales) sufre una secuencia de transformaciones:
 - Unarias (en el caso de las mutaciones)
 - De orden mayor (caso del cruzamiento)

- Los individuos compiten por sobrevivir
- Un mecanismo de selección, en base a ciertas características de los individuos, selecciona la generación siguiente
- Después de una cierta cantidad de generaciones, el programa **converge**, en cuyo caso es de esperar que los individuos resultantes representen la **solución óptima**

INICIOS

- A fines de los 60's y comienzos de los 70's, el concepto de Algoritmo Genético fue desarrollado en la Universidad de Michigan, bajo la dirección de John Holland.



ALGORITMOS GENÉTICOS

- Programas de computador que "evolucionan" de forma que recuerdan la selección natural, pueden resolver problemas complejos y que aún sus creadores no entienden completamente

John H. Holland



ESTRUCTURA DE UN PROGRAMA EVOLUTIVO

Begin

$t \leftarrow 0$

Inicializar $P(t)$

Evaluar $P(t)$

Mientras no sea condición de término

$t \leftarrow t + 1$

Seleccionar $P(t)$ a partir de $P(t-1)$

Recombinar $P(t)$

Evaluar $P(t)$

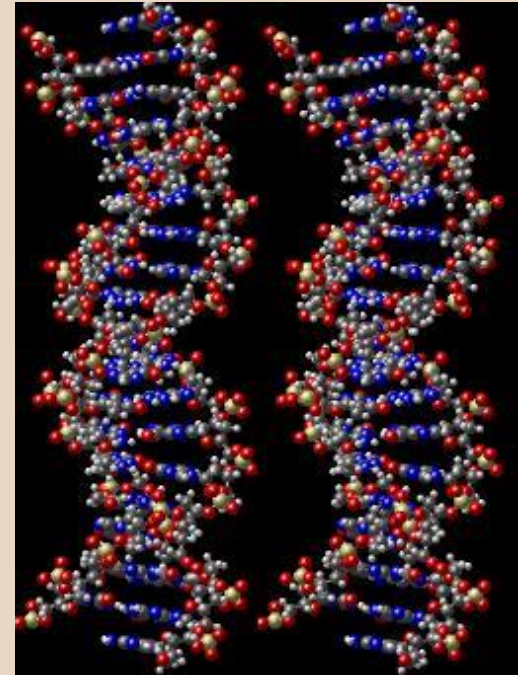
End

- Un programa de evolución es un algoritmo probabilístico que mantiene una población de individuos
- En la iteración t

$$P(t) = \{x_i^t, \dots, x_n^t\}$$

- Cada individuo representa una solución potencial al problema planteado, y es implementado como alguna estructura de datos S (posiblemente compleja)

- Cada solución x_i^t es evaluada para obtener una medida de su *bondad* (fitness)
- Luego, una nueva población (en la iteración $t+1$) se forma seleccionando los individuos *más adecuados* (paso de selección)



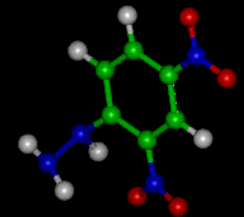
- Algunos miembros de la nueva población sufren transformaciones (paso de recombinación), por medio de operadores genéticos, para formar nuevas soluciones.

MUTACIÓN

- Hay transformaciones unarias, que crean nuevos individuos a partir de pequeños cambios en un solo individuo
- $m : s \rightarrow s$ (mutación)

CRUZAMIENTO

- Hay transformaciones de orden superior que recombinan partes de varios (dos o más individuos) para generar uno nuevo
- $c : s \times s \dots \times s \rightarrow s$ (crossover)



EJEMPLO

- Supongamos que buscamos un grafo que debe satisfacer ciertos requerimientos (por ejemplo, una topología óptima de una red de comunicación o transporte).
- Cada individuo representa una solución potencial al problema, o sea cada individuo es un grafo.

- La población inicial, $P(0)$ es una población de grafos, lista para evolucionar. Normalmente, la función de evaluación es conocida y representa los requerimientos del problema.
- La función de evaluación entrega el *fitness* de cada grafo, distinguiéndose entonces los grafos peores y los mejores

- En este punto es posible considerar diversos operadores de mutación y de cruzamiento. Por ejemplo, la aparición o desaparición de una arista y la combinación de dos o más grafos.
- Si el grafo que buscamos es conexo y acíclico (o sea, un árbol), entonces las operaciones deben mantener esas características que serán consideradas en la función de evaluación.

PENALIZACIONES

- Una de las formas más simples de tratar con la función de evaluación es castigar a aquellos individuos que no cumplan ciertas características. Así, un grafo que no sea un árbol podría recibir, por ejemplo, una nota negativa, lo que incidirá en el proceso de selección.
- Modificación de los operadores genéticos para que los individuos que sean generados cumplan con las características del problema.

- Muchos programas pueden formularse para un problema dado. Estos programas pueden ser diferentes en muchos aspectos
 - Estructuras de datos utilizadas
 - Operadores genéticos considerados
 - Métodos para el manejo de restricciones
 - Parámetros:
 - Tamaño de la población
 - Probabilidad de aplicación de operadores
 - etc.

SIN EMBARGO, LA IDEA CENTRAL PERMANECE

- Una población de individuos sufre transformaciones, y durante este proceso de evolución, los individuos luchan por su supervivencia.

- Una de las ideas actuales es que la representación **natural** de una potencial solución para un problema dado, más una familia de operadores genéticos **aplicables**, puede ser de utilidad en la aproximación a la solución en muchos problemas.

ESTO LLEVA A EXPERIMENTAR CON...

- strings de longitud variable
- estructuras más ricas que los strings binarios
- operadores genéticos modificados

TERMINOLOGÍA

- *Cromosoma*
 - Candidato a solución de un problema
 - A menudo se codifica como un string de bits (0s y 1s)
- *Genes*
 - Un solo bit o bloques simples de bits adyacentes que codifican un elemento particular de la solución candidata
- *Alelo*
 - 0 o 1 en strings de bits

TERMINOLOGÍA (CONT.)

- *Cruzamiento*
 - Intercambio de material genético entre dos cromosomas padres
- *Mutación*
 - Cambio del valor de un bit elegido al azar
- *Genotipo*
- *Fenotipo*
- *Padres*
- *Descendientes*

UN VISTAZO AL FITNESS

- Fitness
 - AGGMCGBL y MGGMCGBL : 1
 - AGGMCDBL y LBMPAFGA : 8
- Puede graficarse como una figura $(l+1)$ -dimensional.

OPERADORES GENÉTICOS

- Selección
 - Selecciona cromosomas en la población para reproducción. A mejor fitness del cromosoma, más veces puede ser seleccionado para reproducción.
- Cruzamiento
 - Se elige aleatoriamente un punto del cromosoma y se intercambian los genes que están antes y después de ese punto entre dos cromosomas para generar descendientes.
- Mutación
 - Cambio aleatorio de los bits en un cromosoma.

UN AG SIMPLE (DETALLE)

1. Se inicia con una población creada aleatoriamente
2. Calcula el **fitness** para cada cromosoma en la población
3. Repite los siguientes pasos hasta crear n descendientes:
 - selecciona un par de cromosomas padres en la población actual
 - con probabilidad p_c cruza el par en un punto elegido randómicamente para formar dos descendientes
 - muta los dos descendientes en cada lugar con probabilidad p_m .

UN AG SIMPLE (DETALLE) (CONT.)

4. Reemplaza la población actual por la nueva población.
5. Ir al paso 2.

Cada iteración de este proceso se llama *generación*

ALGUNAS APLICACIONES DE LOS AGS

- Optimización
- Programación Automática
- Machine Learning
- Economía
- Sistemas inmunes
- Ecología
- Genética poblacional
- Evolución y aprendizaje
- Sistemas sociales

AHORA PODEMOS RESPONDER LA CONSULTA INICIAL: ¿QUÉ SON LOS AGS?

- Son algoritmos estocásticos cuyos métodos de búsqueda modelan un fenómeno natural: la herencia genética y la evolución darwiniana por la supervivencia.

- *La metáfora subyacente en AG es la de la evolución natural.*
- *En la evolución, el problema que cada especie enfrenta es el de buscar las adaptaciones que sean beneficiosas ante un ambiente complicado y cambiante.*
- *El conocimiento que cada especie ha ganado es incorporado en la estructura cromosómica de sus miembros.*

LA TAREA

- El tema y la idea central
- Es un trabajo grupal (no menos de 3 personas, no más de 5)
- Validación del tema: 17 de agosto
- Punto de control inicial: 31 de agosto
- Punto de entrega final: 14 de septiembre

ALGUNAS APLICACIONES

- **Radiología:** Mejora detección de bordes, eliminación de ruido, reducción puntos falsos en MRI, e imágenes con ultrasonido.
- **Oncología:** Análisis de microarreglos y cerosas expresión genética de células cancerosas para clasificar tumores.
- **Terapias de radiación:** Optimiza los ángulos de radiación para proteger tejido sano.
- **Farmacología:** Ayuda en el diseño de medicamentos, docking molecular, y farmacovigilancia a través de la búsqueda de estructuras químicas óptimas.
- **Cardiología:** Mejora en la interpretación de ECGs y detección automática de arritmias.
- **Logística en salud:** Distribución de recursos, programación de turnos y gestión del flujo de pacientes.
- **Pequilar**

ALGUNAS APLICACIONES

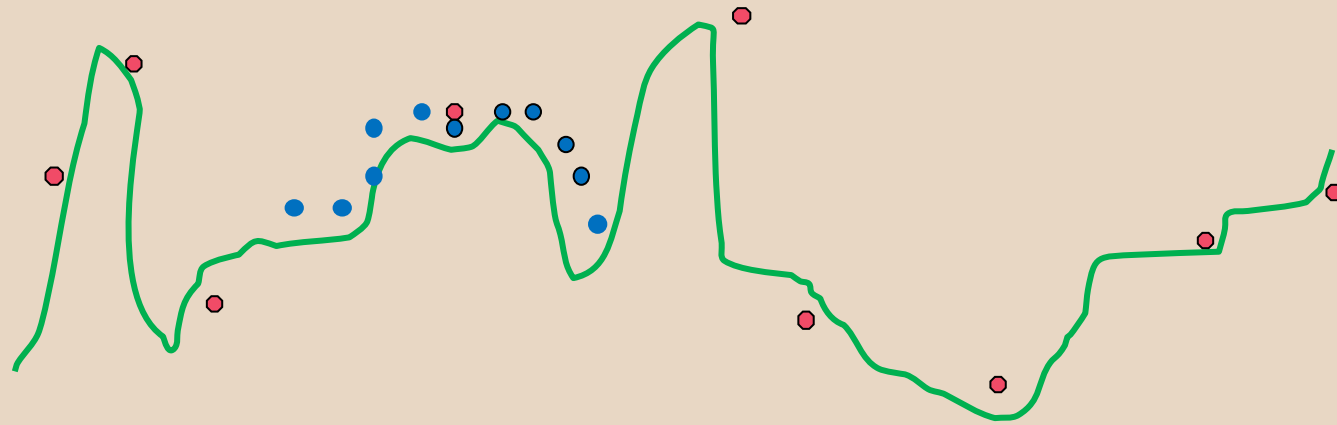
- **Optimización:** De carteras de inversión. Cada individuo representa una combinación de acciones y los porcentajes de capital asignados a esas acciones.
- **Selección natural:** Análisis de microarreglos y expresión genética de células cancerosas para identificación primaria y clasificación
- **Detección fraudes:** identificación de clusters.
- **Predicción de precios:** Optimización de reglas de negociación.

ALGUNAS APLICACIONES

- **Diseño estructural:** Creación de piezas más livianas y resistentes con forma complejas..
- **Control de procesos:** Ajuste automático de variables en fábricas y maquinarias.
- **Logística:** Planificación en la distribución/suministro de bienes.
- **Arquitectura:** Alternativas de adaptación al entorno.

- Un proceso de evolución gatillado para una población de cromosomas corresponde a una búsqueda a través de un espacio de potenciales soluciones.
- Esta búsqueda requiere el balance de dos objetivos aparentemente conflictivos:
 - Hillclimbing
 - Búsqueda randómica

- Hillclimbing. Explotación de las mejores soluciones (***)
- Búsqueda randómica. Exploración del espacio de búsqueda (***)



COMPONENTES DE UN AG

- Para un problema particular, un algoritmo genético debe tener los siguientes cinco componentes

- Una **representación** genética para las potenciales soluciones al problema
- Una forma de crear una **población inicial** de soluciones potenciales
- Una **función de evaluación** (como si fuera el ambiente) que valora las soluciones en función de su fitness
- **Operadores genéticos** que alteren la composición de los hijos
- Valores para los **diferentes parámetros** de un AG (tamaño de la población, probabilidad de aplicación de un operador genético, etc.)

EJEMPLO. OPTIMIZACIÓN DE UNA FUNCIÓN SIMPLE

- Sea la función

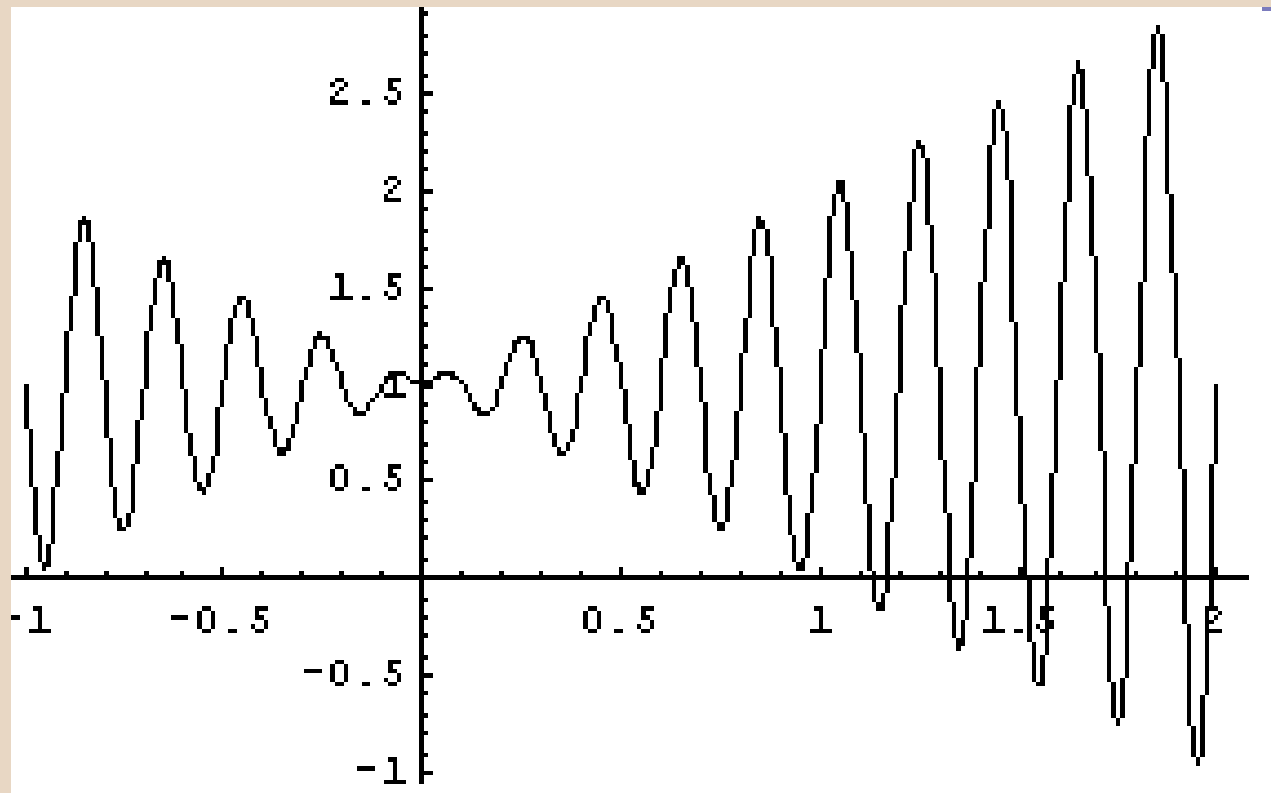
$$f(x) = x \cdot \sin(10\pi \cdot x) + 1.0$$

El problema es encontrar $x \in [-1,2]$ tal que maximice la función f , o sea encontrar x_0 tal que

$$f(x_0) \geq f(x) \quad \forall x \in [-1,2]$$

OPTIMIZACIÓN DE UNA FUNCIÓN

$$f(x) = x \cdot \sin(10\pi \cdot x) + 1.0$$



- Es relativamente fácil analizar esta función.

- Tomando la primera derivada,

$$f'(x) = \sin(10\pi x) + 10\pi x \cdot \cos(10\pi x) = 0$$

- Lo que es equivalente a

$$\tan(10\pi x) = -10\pi x$$

- La ecuación anterior tiene infinitas soluciones.
- La función f alcanza su máximo local para $f(1.85) = 2.85$
- Construyamos un AG para resolver el mismo problema

REPRESENTACIÓN

- Usaremos un vector binario para representar los valores reales de la variable x
- La longitud del vector depende de la precisión que se requiera (sean seis lugares después del punto decimal en este caso)
- Ya que el dominio de la variable tiene longitud 3, el rango debe dividirse en a lo menos $3 \times 1.000.000$ (¿... cuántos bits?)

22 BITS SERÁN SUFICIENTES

- De acuerdo al siguiente criterio
- $2.097.152 = 2^{21} < 3.000.000 \leq 2^{22} = 4.194.304$

MAPEAMIENTO DE BINARIO A REAL

- Se convierte el string binario desde base 2 a base 10:
 - $(\langle b_{21} \ b_{20} \ \dots \ b_0 \rangle)_2 = (\sum_{i=0}^{21} b_i \ 2^i)_{10} = x'$
- Se encuentra el número real x correspondiente
 - $x = -1.0 + x' \cdot (3/(2^{22} - 1))$
 - donde -1.0 es el límite izquierdo del dominio y 3 es la longitud del dominio.

EJEMPLO

- El cromosoma (1000101110110101000111) representa el número 0.637197, ya que
- $x' = (1000101110110101000111)_2 = 2288967$ y $x = -1.0 + 2288967 \cdot (3/4194303) = 0.637197$
- Naturalmente, (000000000000000000000000) y (111111111111111111111111) representan los extremos del dominio, -1.0 y 2.0 respectivamente

POBLACIÓN INICIAL

- Se crea una población de cromosomas donde cada uno de ellos es un vector binario de 22 bits
- Los 22 bits para cada uno de los cromosomas se inicializan en forma randómica

FUNCIÓN DE EVALUACIÓN

- La **función de evaluación eval** para vectores binarios **v** es equivalente a la función f
- $\text{eval}(v) = f(x)$
- donde el cromosoma **v** representa el valor real **x**

CROMOSOMAS Y FITNESS

- $v_1 = (1000101110110101000111)$

- $x_1 = 0.637197$

- $v_2 = (0000001110000000010000)$

- $x_2 = -0.958973$

- $v_3 = (1110000000111111000101)$

- $x_3 = 1.627888$

RANKING

- $\text{eval}(v_1) = f(x_1) = 1.586345$
- $\text{eval}(v_2) = f(x_2) = 0.078878$
- $\text{eval}(v_3) = f(x_3) = 2.250650$

Claramente, v_3 es el cromosoma mejor evaluado

OPERADORES GENÉTICOS

- Existen dos **operadores genéticos** relevantes:
 - Mutación
 - Cruzamiento
- La mutación altera uno o más genes (posiciones en un cromosoma) con una probabilidad igual a la **razón de mutación**

$$v_3 = (1110000000111111000101)$$

• $x_3 = 1.627888$

- Supongamos que se elige el quinto gen en el cromosoma v_3 para mutación.
- Ya que ese gen tiene el valor cero, su valor es cambiado a 1, por lo que después de la mutación tenemos $v_3' = (1110100000111111000101)$
- El cromosoma representa el valor $x_3' = 1.721638$ y $f(x_3') = -0.082257$
- Esto significa que esta mutación particular lleva a una disminución significativa del fitness del cromosoma v_3'

$$v_3 = (1110000000111111000101)$$

• $x_3 = 1.627888$

- Por otra parte, si el gen seleccionado para mutación es el número 10, entonces
 - $v_3'' = (1110000001111111000101)$
- Así, a x_3'' le corresponde el valor 1.630818 y a $f(x_3'')$ el valor 2.343555
- O sea, se ha conseguido mejorar el valor del fitness

- Supongamos que el punto de cruzamiento ha sido elegido (randómicamente) después del quinto gen

- $V_2 = (0000001110000000010000)$

- $V_3 = (1110000000111111000101)$

- $v_2' = (0000000000111111000101)$

- $v_3' = (1110001110000000010000)$

- $f(v_2') = f(-0.998113) = 0.940865$

- $f(v_3') = f(1.666028) = 2.459245$

- El segundo hijo es mejor evaluado que sus dos padres

PARÁMETROS

- Para este problema particular se usaron los siguientes parámetros
 - Tamaño de la población: 50
 - Probabilidad de cruzamiento: 0.25
 - Probabilidad de mutación: 0.01

RESULTADOS

- El mejor cromosoma después de 150 generaciones fue
 - $v_{\max} = (1111001101000100000101)$
- Que corresponde al valor $x_{\max} = 2.850773$
- $f_{\max}$ es ligeramente superior a 2.85

1	1.441942
6	2.250003
8	2.250283
9	2.250284
10	2.250363
12	2.328077
39	2.344251
40	2.345087
51	2.738930
99	2.849246
137	2.850217
145	2.850227

¿CÓMO TRABAJAN LOS AGS?

- Suposiciones
 - Problemas de maximización
 - La función objetivo toma valores positivos en su dominio

- Supongamos que deseamos maximizar una función de k variables
 - $f(x_1, \dots, x_k) : \mathbb{R}^k \rightarrow \mathbb{R}$
- Supongamos además que cada variable x_i puede tomar valores de un dominio
 - $D_i = [a_i, b_i] \subseteq \mathbb{R}$ y $f(x_1, \dots, x_k) > 0, \forall x_i \in D_i$

- Supongamos que queremos 6 valores decimales
 - Cada dominio D_i debe ser dividido en $(b_i - a_i) \cdot 10^6$ rangos de igual tamaño
 - Sea m_i el menor entero tal que $(b_i - a_i)10^6 \leq 2^{m_i} - 1$
- Entonces, una representación que tenga cada variable x_i codificada como un string binario de longitud m_i , claramente satisface el requerimiento de precisión

- $x_i = a_i + \text{decimal}(10\dots101_2) \cdot (b_i - a_i) / (2^{m_i} - 1)$
- Esta fórmula interpreta cada string donde $\text{decimal}(\text{string}_2)$ representa el valor decimal de ese string binario

- Cada cromosoma (como una solución potencial) es representado por un string binario de longitud $m = \sum m_i$; ($i = 1 \dots k$)
- Los primeros m_1 bits mapean un valor en el rango $[a_1, b_1]$, el próximo grupo de m_2 bits mapean sobre un valor en el rango $[a_2, b_2]$, ..., el último grupo de m_k bits mapea sobre un valor en el rango $[a_k, b_k]$

INICIALIZACIÓN DE LA POBLACIÓN

- Para inicializar la población podemos usar un random
- Si se tiene información sobre la distribución de óptimos potenciales, podemos usar tal información para “acomodar” el conjunto inicial de soluciones (potenciales)

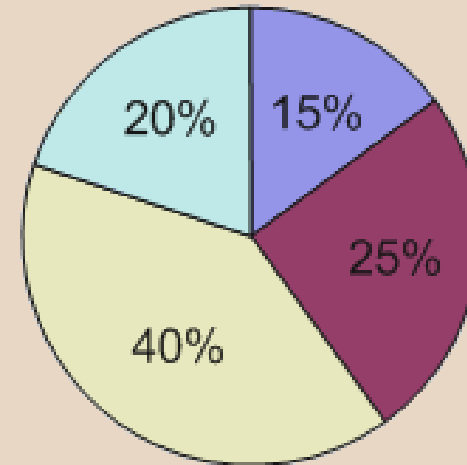
- El resto del algoritmo es simple:
 - En cada generación se evalúa cada cromosoma (usando la función f).
 - Se selecciona una nueva población con respecto a la probabilidad de distribución basada en los valores de los fitness.
 - Se alteran cromosomas en la nueva población por mutación y cruzamiento.

- Después de un cierto número de generaciones, cuando no se observa un mejoramiento, los mejores cromosomas representan una solución optimal (posiblemente la global).
- A menudo el algoritmo es detenido después de un número fijo de iteraciones, dependiendo de los criterios de velocidad y recursos (computacionales).

SELECCIÓN (USANDO RULETA)

- Distribución de probabilidad basada en el fitness

1	000110110	15%
2	111001101	25%
3	000110110	40%
4	111101111	20%



CONSTRUCCIÓN DE RULETA (VALORES DE FITNESS POSITIVOS)

- Calcular el valor del fitness $\text{eval}(v_i)$ para cada cromosoma v_i ($i= 1, \dots, \text{tam_pobl}$)
- Encontrar el fitness total de la población: $F = \sum \text{eval}(v_i) \quad (i=1, \dots, \text{tam_pobl})$
- Calcular la probabilidad de una selección p_i para cada cromosoma v_i : $p_i = \text{eval}(v_i)/F$
- Calcular la probabilidad acumulativa q_i para cada cromosoma v_i : $q_i = \sum p_j \quad (j =1, \dots, i)$

EL PROCESO DE SELECCIÓN

- Se basa en el giro de la ruleta: se hace girar `tam_pobl` veces, seleccionando cada vez un solo cromosoma para una nueva población
- Se genera un número random r en $[0,1]$
- Si $r < q_1$, entonces se selecciona el primer cromosoma, v_1 ; en caso contrario se selecciona el i -ésimo cromosoma v_i , tal que $q_{i-1} < r \leq q_i$

- Algunos cromosomas pueden ser seleccionados más de una vez
- Percepción de supervivencia:
 - Los mejores se reproducen más
 - Los casos “promedio” se mantienen
 - Los peores mueren

PROBABILIDAD DE CRUZAMIENTO P_c

- Esta probabilidad nos da el número esperado de cromosomas que participarán en la operación de cruzamiento: $p_c \cdot \text{tam_pobl}$
- Operación. Para cada cromosoma en la (nueva) población:
 - Genere un número random r en $[0,1]$
 - Si $r < p_c$, seleccione el cromosoma dado para cruzamiento

CRUZAMIENTO

- Se seleccionan aleatoriamente las parejas
- Para cada par de cromosomas emparejados, se genera un número random **pos** en el rango $[1, m-1]$
- El número **pos** indica la posición del punto de cruzamiento

EJEMPLO

- Dos cromosomas
 - $(b_1 b_2 \dots b_{\text{pos}} b_{\text{pos}+1} \dots b_m)$ y
 - $(c_1 c_2 \dots c_{\text{pos}} c_{\text{pos}+1} \dots c_m)$
- Son reemplazados por el par de descendientes
 - $(b_1 b_2 \dots b_{\text{pos}} c_{\text{pos}+1} \dots b_m)$ y
 - $(c_1 c_2 \dots c_{\text{pos}} b_{\text{pos}+1} \dots b_m)$

PROBABILIDAD DE MUTACIÓN P_M

- Nos da el número esperado de bits mutados $p_m \cdot m \cdot \text{tam_pobl}$
- Todos los bits (en cada cromosoma, en toda la población) tienen igual probabilidad de sufrir una mutación (cambiar un 0 por un 1 o vice-versa)

MUTACIÓN

- Por cada cromosoma en la población actual (o sea, después del cruzamiento) y para cada bit dentro del cromosoma:
 - Generar un número random r en el rango $[0,1]$
 - Si $r < p_m$, mutar el bit

FINALE PRESTO

- Luego de la selección, el cruzamiento y la mutación, la nueva población está lista para su próxima evaluación
- Se construye nueva ruleta de acuerdo a los valores actuales de los fitness
- El resto de la evolución es, simplemente, la repetición cíclica de los pasos anteriores

EJEMPLO: MAX_UNOS

Objetivo: maximizar el número de 1s en un string de longitud 5, compuesto solamente de 1s y 0s

⇒ *tamaño de la población* = 4

longitud del cromosoma = 5

función de fitness = número de genes que son 1

POBLACIÓN EJEMPLO

individuo 1:

cromosoma = 11001

fitness = 3

individuo 3:

cromosoma = 11111

fitness = 5

individuo 2:

cromosoma = 00001

fitness = 1

individuo 4:

cromosoma = 01110

fitness = 3

REPRODUCCIÓN

- Consiste de
 - Selección
 - mating pool (conjunto de pades candidatos para la próxima generación es normalmente el tamaño de la población)
 - posiblemente más de una copia de algunos individuos
 - Cruzamiento (cross-over)
 - Mutación

SELECCIÓN

- Usa selección por ruleta
 - fitness proporcional
 - El número esperado de representantes de cada individuo es proporcional a su fitness
- *pop.size* es el tamaño de la población

$$prob_i = \frac{fitness_i}{\sum_j fitness_j}, j = 1 \dots pop.size$$

EJEMPLO DE SELECCIÓN

Población actual

i1: 11001, 3

i2: 00001, 1

i3: 11111, 5

i4: 01110, 3

Probabilidad de selección:

$$\text{prob}(i1) = 3/12 = 0.25$$

$$\text{prob}(i2) = 1/12 = 0.08$$

$$\text{prob}(i3) = 5/12 = 0.42$$

$$\text{prob}(i4) = 3/12 = 0.25$$

Copias por individuo:

i1: $(3/12 \cdot 4)$ 1

i2: $(1/12 \cdot 4)$ 0

i3: $(5/12 \cdot 4)$ 2

i4: $(3/12 \cdot 4)$ 1

Suposición:

la ruleta se gira 4 veces

1 copia de i1

2 copias de i3

1 copia de i4

Ellos son copiados en el *mating pool*

EJEMPLO PAREJAS

Candidatos actuales:

individuo 1: 11001 (*i1*)

individuo 2: 11111 (*i3*)

individuo 3: 11111 (*i3*)

individuo 4: 01110 (*i4*)

Pares:

Par 1:

11001

11111

Par 2:

11111

01110

Suposición:

(individuo 1, individuo 3)

y

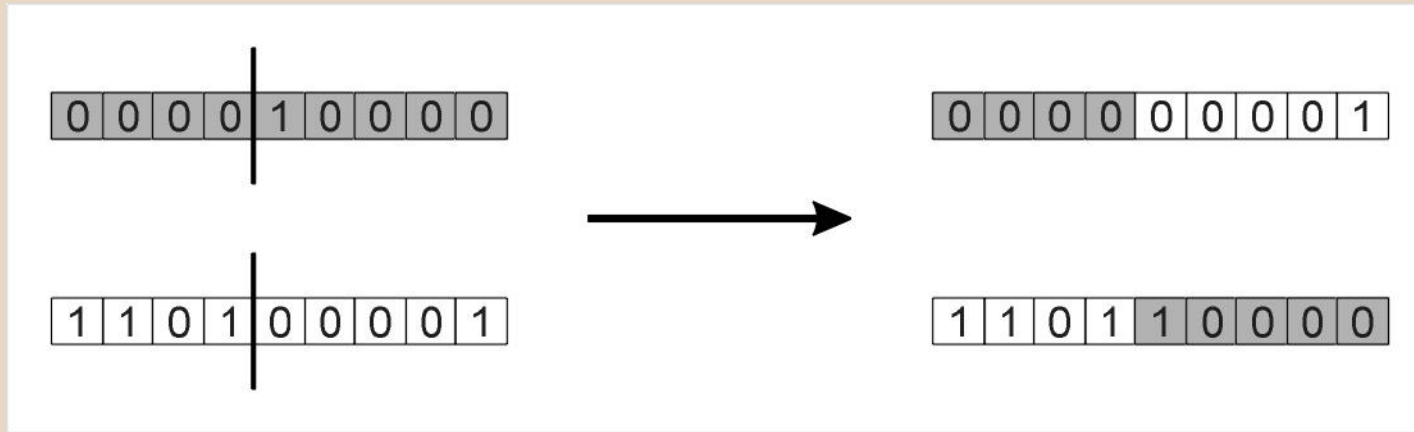
(individuo 2, individuo 4)

Son candidatos a parejas para reproducción.

RECOMBINACIÓN

- nuevos individuos formados desde parejas de padres
- un punto de cruzamiento
- probabilidad de cruzamiento: p_c
 - a.k.a. tasa de cruzamiento
 - típicamente en el rango $[0.5, 1.0]$

CRUZAMIENTO EN UN PUNTO



EJEMPLO DE CRUZAMIENTO

Supongamos $p_c=1.0$

Para el primer par:

Punto de cruzamiento: 3

110 | 01 → 11011

111 | 11 → 11101

Para el segundo par:

Punto de cruzamiento: 1

1 | 1111 → 11110

0 | 1110 → 01111

los nuevos individuos:

i1: 11011

i2: 11101

i3: 11110

i4: 01111

MUTACIÓN

- probabilidad de mutación: p_m
 - a.k.a. tasa de mutación
 - igual probabilidad para cada gen
 - para un cromosoma de longitud L ; número esperado de cambios: $L * p_m$
 - típicamente es un valor pequeño (2 a 6%)
 - depende de la naturaleza del problema

MUTACIÓN

1	0	1	0	0	0	0	1	0
---	---	---	---	---	---	---	---	---



1	0	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---	---

EJEMPLO DE MUTACIÓN

Supongamos:

como resultado de un random

primer gen de i1

y

cuarto gen de i3

se eligen para mutación

i1: 11011 → 01011

i3: 11110 → 11100

DINÁMICA POBLACIONAL

- AG generacional
 - No se superponen las poblaciones
 - Los descendientes reemplazan a los padres

EJEMPLO DE NUEVA POBLACIÓN

individuo 1:

cromosoma = 01011

fitness = 3

individuo 2:

cromosoma = 11101

fitness = 4

individuo 3:

cromosoma = 11100

fitness = 3

individuo 4:

cromosoma = 01111

fitness = 4

CRITERIO DE DETENCIÓN

- El ciclo principal se repite hasta que se cumpla el criterio de detención:
- Para un número predeterminado de generaciones
- Hasta que se alcance el objetivo
- Hasta que la población converge

CONVERGENCIA

- Progresión hacia la uniformidad
- *Convergencia genética*: cuando el 95% de los individuos tienen el mismo valor para ese gen
- *Convergencia poblacional*: cuando todos los genes han alcanzado la convergencia
- El fitness promedio se aproxima al mejor valor

EJEMPLOS DE CRITERIOS DE DETENCIÓN

Caso 1:

Número de generaciones = 250

- Ciclo repetido 250 veces
- Se encuentra el mejor individuo en cada generación
- El mejor individuo se transforma en la solución

Caso 2:

objetivo: maximizar el número de 1s

meta: encontrar el individuo que tiene 1s en todas las ubicaciones de genes

- Ciclo repetido eternamente
- Se encuentra el mayor individuo de cada generación
- Termina cuando se encuentra el individuo objetivo

CRITERIOS DE ELECCIÓN

- a.k.a. reemplazo
- determina quien sobrevive en la próxima generación
 - reduce $(m+1)$ a m
 - m tamaño de la población (a.k.a. número de padres)
 - 1 número de descendientes al final de una generación
- hay varias estrategias de reemplazo

REEMPLAZO BASADO EN LA EDAD

- fitness no se toma en cuenta
- cada individuo existe un cierto número de generaciones
 - en AGs solamente por 1 generación
- e.g. crea 1 descendiente y lo inserta en la población en cada generación
 - FIFO
 - reemplazo random (no se recomienda)

REEMPLAZO BASADO EN EL FITNESS

- Usa el fitness para seleccionar m individuos desde $(m+l)$ (m padres, l descendientes)
 - selección de padres basada en fitness
 - reemplazo de los peores
 - mejoramiento rápido en calidad de los individuos in
 - posible convergencia prematura
 - elitismo
 - mantiene los mejores en la población
 - reemplaza un individuo (peor, similares, etc)

¿POR QUÉ FUNCIONAN LOS AGS?

- Los fundamentos teóricos de los AGs descansan en la representación de soluciones sobre strings binarios y en la noción de esquema.
- Un esquema se construye por medio de la introducción de un símbolo **don't care** (*) en el alfabeto genético.

- Un esquema representa a todos los strings (un hiperplano o subconjunto del espacio de búsqueda) que coinciden en todas aquellas posiciones en que no hay un símbolo *
- Por ejemplo, el esquema (*111100100) parea dos strings:
(1111100100) y (0111100100)

- El “esquema” (1001110001) representa a un solo string (el mismo)
- El esquema (*****) representa a todos los strings de longitud 10
- Si r es el número de símbolos don't care, cada esquema parea exactamente 2^r strings
- Cada string de longitud m es pareado por 2^m esquemas

PROPIEDADES DE UN ESQUEMA

- Orden
- Longitud de definición

ORDEN DE UN ESQUEMA

- Es el número de posiciones fijas (0 o 1) presentes en el esquema, y se denota $o(S)$
- O sea, es la longitud de la plantilla menos el número de símbolos **don't care**
- El orden define la especialidad de un esquema

LONGITUD DE DEFINICIÓN DEL ESQUEMA

- Es la distancia entre la primera y la última posición fija del esquema
- Define el grado de compactación de la información contenida en el esquema
- Se denota por $\delta(S)$

EJEMPLO

- $S_1 = (**001*110)$
- $S_2 = (****00**0*)$
- $S_3 = (11101**001)$

- $o(S_1) = 6, \quad o(S_2) = 3, \quad o(S_3) = 8;$

- $\delta(S_1) = 10 - 4 = 6, \quad \delta(S_2) = 9 - 5 = 4, \quad \delta(S_3) = 10 - 1 = 9$

- Cruzamiento y mutación pueden destruir y crear instancias de un esquema.
- Un esquema S sobrevive al cruzamiento si uno de los descendientes también es una instancia de él.
- La probabilidad de sobrevivir bajo **cruzamiento** es mayor para esquemas de longitud **más corta**.
- La probabilidad de sobrevivir bajo **mutación** es mayor para esquemas de **orden bajo**.

EJEMPLO

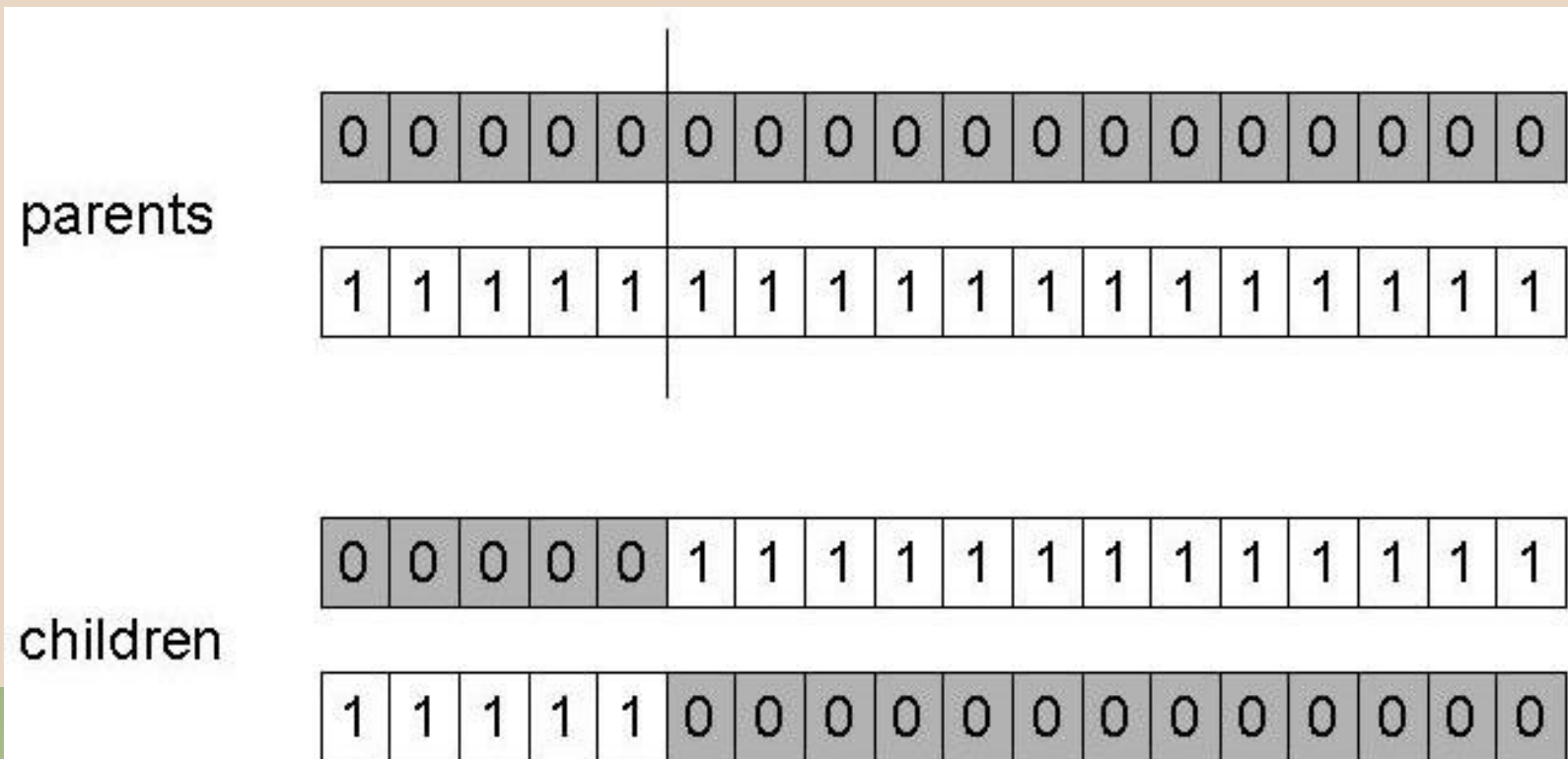
- Por favor, traten de acompañar el desarrollo

ALGORITMO GENÉTICO SIMPLE

1. Seleccionar padres para cruzar
(Miembros de la población candidatos a cruzamiento = todos)
2. Revolver (como en un juego de cartas)
3. Para cada par consecutivo aplique cruzamiento con probabilidad p_c , si no hay cruzamiento, copiar los padres
4. Para cada descendiente, aplique mutación (cambiar el bit con probabilidad p_m independiente para cada bit)
5. Reemplace toda la población con los *nuevos* individuos

OPERADORES: CRUZAMIENTO (SIMPLE)

- Elección de un punto random en el cromosoma
- Crear dos hijos de acuerdo a la división recién determinada
- P_c típicamente en el intervalo (0.6, 0.9)



OPERADORES: MUTACIÓN

- Alterar cada gen independientemente con una probabilidad p_m
- p_m se llama tasa de mutación
 - Típicamente entre $1/(\text{tamaño_población})$ y $1/(\text{largo_cromosoma})$

parent

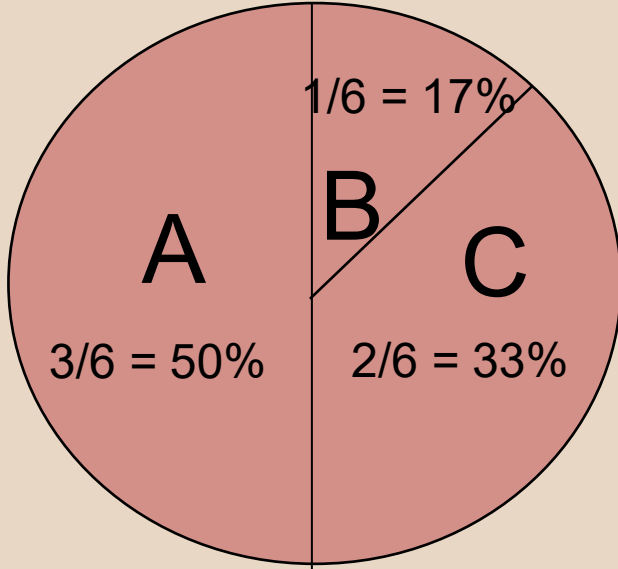
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

child

0	1	0	0	1	0	1	1	0	0	0	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

OPERADORES: SELECCIÓN

- Idea: los mejores individuos tienen más probabilidades
 - Proporcional al fitness
 - Implementación: técnica de la ruleta
 - Asignar a cada individuo una parte de la ruleta
 - Girar la ruleta n veces para seleccionar n individuos



$\text{fitness}(\text{A}) = 3$

$\text{fitness}(\text{B}) = 1$

$\text{fitness}(\text{C}) = 2$

EJEMPLO (GOLDBERG)

- Problema: maximizar x^2 sobre $\{0, 1, \dots, 31\}$
- Aproximación usando AGs:
 - Representación: codificación binaria, por ejemplo $01101 \leftrightarrow 13$
 - Tamaño de población: 4
 - Selección a través de ruleta
 - Inicialización: random
- Se muestra un ciclo generacional

SELECCIÓN

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$	Expected count	Actual count
1	0 1 1 0 1	13	169	0.14	0.58	1
2	1 1 0 0 0	24	576	0.49	1.97	2
3	0 1 0 0 0	8	64	0.06	0.22	0
4	1 0 0 1 1	19	361	0.31	1.23	1
Sum			1170	1.00	4.00	4
Average			293	0.25	1.00	1
Max			576	0.49	1.97	2

CRUZAMIENTO

String no.	Mating pool	Crossover point	Offspring after xover	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 1	4	0 1 1 0 0	12	144
2	1 1 0 0 0	4	1 1 0 0 1	25	625
2	1 1 0 0 0	2	1 1 0 1 1	27	729
4	1 0 0 1 1	2	1 0 0 0 0	16	256
Sum					1754
Average					439
Max					729

MUTACIÓN

String no.	Offspring after xover	Offspring after mutation	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 0	1 1 1 0 0	26	676
2	1 1 0 0 1	1 1 0 0 1	25	625
2	1 1 0 1 1	1 1 0 1 1	27	729
4	1 0 0 0 0	1 0 1 0 0	18	324
Sum				2354
Average				588.5
Max				729

ALGUNAS LIMITACIONES

- Representación es restringida
- Mutación & cruzamiento solo es applicable para strings de bits & representaciones enteras
- Mecanismos de selección son sensibles a poblaciones convergentes con valores cercanos de fitness
- Se puede introducir mejoras

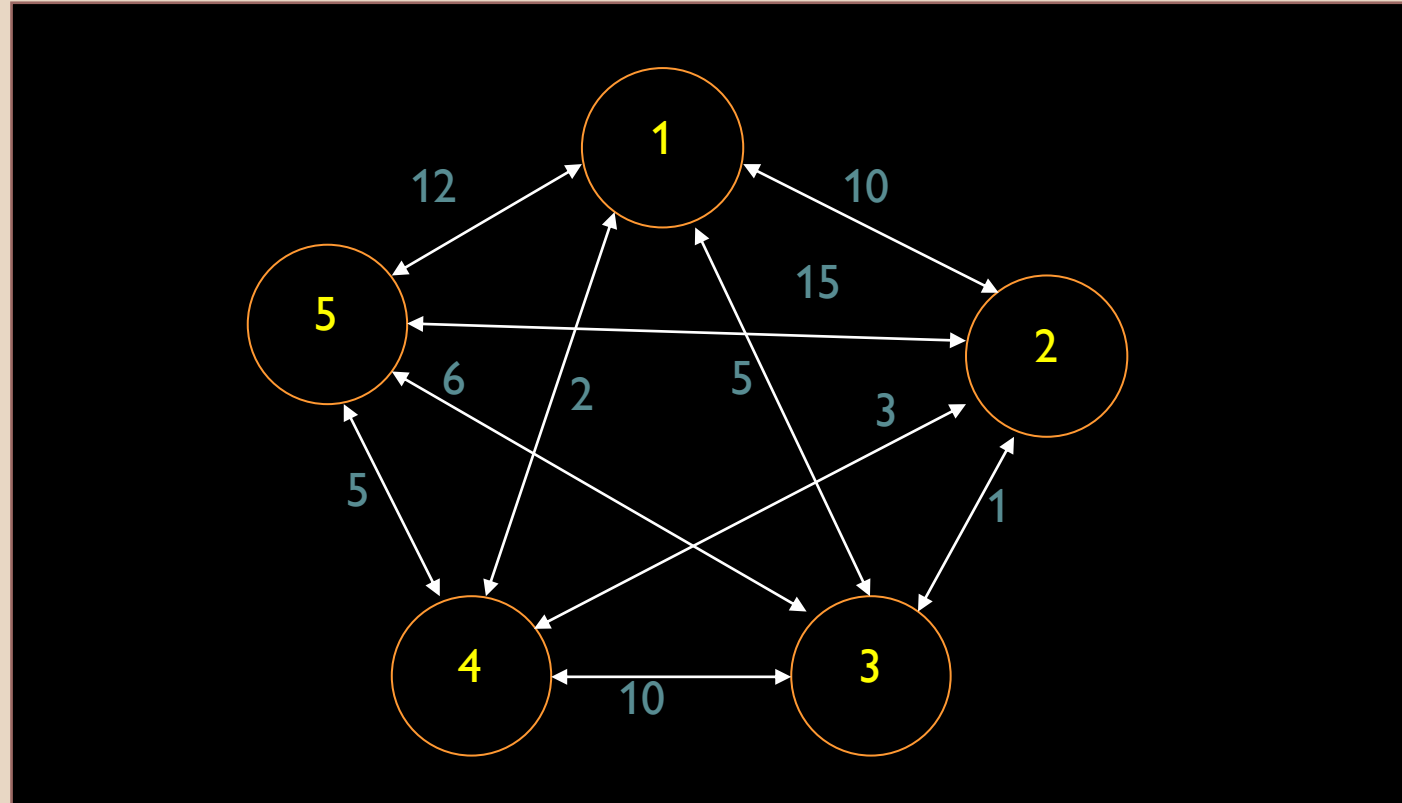
A TRAVES DE OPERADORES ALTERNATIVOS

- Por ejemplo OX

PROBLEMA DEL VENDEDEDOR VIAJERO (TSP)

- El vendedor viajero debe visitar cada ciudad de su territorio exactamente una vez y luego regresar a su punto de partida.
- Dado el costo de viaje entre cada una de las ciudades, él debería planificar su itinerario de manera que el costo total sea mínimo.
- Este es un problema de optimización combinatoria que aparece en numerosas aplicaciones

EJEMPLO



REPRESENTACIÓN

- ¿deberíamos adoptar que el cromosoma sea un vector de enteros?
- ¿deberíamos transformar cada entero a su equivalente en binario?
- ¿cuáles son las ventajas y problemas de cada una de estas representaciones?

REPRESENTACIÓN

- La representación más natural es el vector de enteros
- $v = (i_1 i_2 \dots i_m)$ representa un itinerario: desde i_1 a i_2 , etc, desde i_{m-1} a i_m y regresa a i_1
- v es una permutación de $(1 \ 2 \ \dots \ m)$
- ejemplos:
 - $v_1 = (41523)$
 - $v_2 = (13542)$
 - $v_3 = (45213)$
 - $v_4 = (35412)$

POBLACIÓN INICIAL

- Para el proceso de inicialización se puede usar alguna heurística
 - aceptar unas pocas salidas de algún algoritmo para el TSP, partiendo de diferentes ciudades
- se puede inicializar la población creando un algoritmo que genere aleatoriamente permutaciones de $(1\ 2\ \dots\ m)$

FUNCIÓN DE EVALUACIÓN

- La evaluación del cromosoma está dada por la suma de los costos de viajar entre las ciudades
- ejemplo
 - $v1 = (41523)$ $f(v1) = 2 + 12 + 15 + 1 + 10 = 40$
 - $v2 = (13542)$ $f(v2) = 5 + 6 + 5 + 3 + 10 = 29$
 - $v3 = (45213)$ $f(v3) = 5 + 15 + 10 + 5 + 10 = 45$
 - $v4 = (35412)$ $f(v4) = 6 + 5 + 2 + 10 + 1 = 24$

EL EJEMPLO FINAL

- El problema del ciclista (Mitchel Blancas, U. Trujillo, Perú)

REPASO ESTRUCTURAL

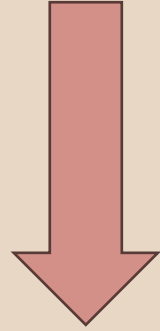
- **Inicio**: Generar aleatoriamente n cromosomas (soluciones posibles)
- **Fitness**: Evaluar $f(x)$ para cada cromosoma x en la población
- **Descendencia**: Crear nueva población
 - Selección: seleccionar cromosomas de acuerdo a $f(x)$ (mejor fitness, mejor oportunidad de ser elegido)
 - **Cruzamiento**: Cruzar cromosomas con cierta probabilidad, si la probabilidad es cero, mantener padres
 - **Mutación**; Con probabilidad p_m , modificar un descendiente
 - **Aceptación**: los descendientes son la nueva población
- **Reemplazo**: Con la nueva población, ejecutar otro ciclo del algoritmo
- **Test**: Si se satisface condición de término, parar (entregar mejor solución)
- **Ciclo**: Volver al paso *Descendencia*

EL PROBLEMA DEL CICLISTA

- Un ciclista debe llevar en su mochila algunos alimentos, para poder asegurar un mínimo de 2000 calorías para un recorrido de 4 horas. La mochila sólo puede llevar un peso máximo de 2.0 kilogramos. La información asociada a los alimentos disponibles es la siguiente:

Número	Artículo	Calorías	Peso (kg)
1	Leche de soya	500	0,5
2	Galleta integral	300	0,1
3	Agua mineral	100	0,5
4	Sandwich de pollo	700	0,25
5	Huevo duro	300	0,15
6	Nueces	400	0,15
7	Yoghurt	500	0,5
8	Manzana	400	0,3

CÓMO PODEMOS PASAR DE LA TABLA ANTERIOR A UN CROMOSOMA



1	1	0	0	1	1	1	0
---	---	---	---	---	---	---	---

- El ciclista lleva leche de soya, galleta integral, huevo duro, nueces y yoghurt

CONTINUACIÓN...

- Población: 4 individuos
- Mínimo de calorías: 2000
- Peso máximo: 2.0 kg
- Para la primera generación tenemos:

Cromosoma	Peso	Calorías	Fitness Peso	Fitness Calorías	Fitness Total
11001110	1.4	2000	23%	21%	44%
01011111	1.45	2600	24%	27%	51%
11011101	1.45	2600	24%	27%	51%
01111011	1.8	2300	30%	24%	54%

CONTINUACIÓN

Cromosoma	Fitness Total	Selección
1 1 0 0 1 1 1 0	44%	0 1 1 1 1 0 1 1
0 1 0 1 1 1 1 1	51%	0 1 0 1 1 1 1 1
1 1 0 1 1 1 0 1	51%	1 1 0 1 1 1 0 1
0 1 1 1 1 0 1 1	54%	0 1 1 1 1 0 1 1

Cruzamiento
0 1 0 1 1 1 1 1
0 1 1 1 1 0 1 1
1 1 0 1 1 0 1 1
0 1 1 1 1 1 0 1

Mutación
0 1 0 1 1 1 1 1
0 1 1 1 1 0 1 0
1 1 0 1 1 1 1 1
0 0 1 1 1 1 0 1

CONTINUACIÓN...

- Población: 4 individuos
- Mínimo de calorías: 2000
- Peso máximo: 2.0 kg

- Para la segunda generación tenemos:

Cromosoma	Peso	Calorías	Fitness Peso	Fitness Calorías	Fitness Total
0 1 0 1 1 1 1 1	1.45	2600	23%	21%	44%
0 1 1 1 1 0 1 0	1.50	1900	24%	27%	51%
1 1 0 1 1 1 1 1	1.95	3100	24%	27%	51%
0 0 1 1 1 1 0 1	1.35	1900	30%	24%	54%

X

X

CONTINUACIÓN...

- Población: 4 individuos
- Mínimo de calorías: 2000
- Peso máximo: 2.0 kg
- Para la segunda generación tenemos:

Cromosoma	Peso	Calorías	Fitness Peso	Fitness Calorías	Fitness Total
0 1 0 1 1 1 1 1	1.45	2600	23%	21%	44%
1 1 0 1 1 1 1 1	1.95	3100	24%	27%	51%
0 1 0 1 1 1 0 1	0.95	2100	24%	27%	51%
0 0 1 1 1 0 1 1	1.7	2000	30%	24%	54%

6.05

9800

OTRO EJEMPLO

OPTIMIZACIÓN DE UNA FUNCIÓN

Objetivo:

Encontrar el conjunto de enteros $\mathbf{x}_i$ que maximicen la función $\mathbf{f}$.

$$\mathbf{f}(\mathbf{x}_i) = \sum_i \mathbf{x}_i^2 \quad i=1, 2, 3$$

$$-512 < \mathbf{x}_i \leq 512$$

OPTIMIZACIÓN DE UNA FUNCIÓN

Representación:

- 1024 enteros en el intervalo dado
- Se necesitan 10 bits

¿Por qué?

0	:	0000000000	(-511)
1	:	0000000001	(-510)
2	:	0000000010	(-509)
...		...	
1023	:	1111111111	(512)

OPTIMIZACIÓN DE UNA FUNCIÓN

Individuo:

- La función tiene 3 parámetros:

$$x_1, x_2, x_3 \quad (-512 < x_i \leq 512)$$

- 10 bits para cada x_i
- cromosoma tiene 30 bits

OPTIMIZACIÓN DE UNA FUNCIÓN

Ejemplo de cromosoma:

11001010101100000000000000110

x_1

x_2

x_3

$$x_1 = (810 - 511) = 299$$

$$x_2 = (768 - 511) = 257$$

$$x_3 = (6 - 511) = -505$$

$$\begin{aligned}\text{fitness} &= (299)^2 + (257)^2 + (-505)^2 \\ &= 410475\end{aligned}$$

OPTIMIZACIÓN DE UNA FUNCIÓN

¿ Y si $\mathbf{x}_i$ fueran números reales?

intervalo: $-5.12 < x_i \leq 5.12$

- Es posible seguir usando números binarios
 - precisión de 2 dígitos después del decimal
 - usa 1024 enteros diferentes (divide el número por 100)
- Usar otras representaciones (e.g. reales)

PROBLEMA DE LA MOCHILA

Objetivo:

$$\max \sum_i v_i * x_i \quad i = 1, 2, \dots, item_count$$

$$subject\ to \quad \sum_i w_i * x_i \leq W$$

$x_i = 0 / 1$ (indica si el ítem i está o no en la mochila)

PROBLEMA DE LA MOCHILA

Ejemplo conjunto objetos:

(1) $w= 2, v=10$

(2) $w= 6, v= 3$

(3) $w=10, v= 8$

(4) $w= 7, v=16$

(5) $w= 4, v=25$

Ejemplo de soluciones factibles:

items: $\{1,2,5\} \Rightarrow$ peso=12
valor= 38

items: $\{3\} \Rightarrow$ peso=10
valor= 8

items: $\{4,5\} \Rightarrow$ peso=11
valor= 41

items: $\{2\} \Rightarrow$ peso=6
valor= 3

Ejemplo de mochila con una capacidad: $W = 12$

PROBLEMA DE LA MOCHILA

Representación:

5 ítems $\Rightarrow$ longitud del cromosoma 5

Ejemplo de cromosomas:

11001 $\Rightarrow$ ítems {1,2,5} se incluyen

00100 $\Rightarrow$ ítems {3} se incluyen

00011 $\Rightarrow$ ítems {4,5} se incluyen

01000 $\Rightarrow$ ítems {2} se incluyen

PROBLEMA DE LA MOCHILA

- ¿Podemos tomar el fitness como el peso total del subconjunto?
 - ¿qué pasa con el sobrepeso?
- ¿cómo manejar los subconjuntos con sobrepeso?
 - ¿los eliminamos? ¿cómo?
 - los penalizamos? ¿cuánto?
 - ¿los podemos corregir?

RELACIÓN AGS Y PROBLEMAS

- Diseñar una representación.
Decidir como inicializar la población.
- Diseñar una correspondencia entre el genotipo y fenotipo.
Diseñar una forma de evaluar al individuo.
Diseñar un operador de mutación adecuado.
Diseñar un operador de cruce adecuado.
- Decidir la selección de individuos para ser padres. Decidir como reemplazar a los individuos
Decidir condición de parada.

CONSIDERACIONES GENERALES PARA LA OPERACIÓN CON ALGORITMOS GENÉTICOS

TRANSFORMACIONES DE LA FUNCIÓN OBJETIVO

En muchos problemas el objetivo es MINIMIZAR una función costo ($g(x)$), en vez de maximizar una función utilidad.

La función adaptación debe ser siempre ≥ 0

puede ser:

- máximo valor $g(x)$ observado hasta ahora
- máximo valor $g(x)$ en la población actual
- máximo valor $g(x)$ en las k últimas generaciones
- ...

REGULACIÓN DEL NÚMERO DE COPIAS

La regulación del número de copias es especialmente importante en AG de pequeñas poblaciones.

Al comenzar a correr AG es común tener unos pocos individuos extraordinarios.

Si se permite la regla de selección normal ($p_{\text{selección}} = f_i / \sum f_i$), los individuos extraordinarios eliminarían una proporción significativa de la población en solo una generación y esto es indeseable, es la principal causa de convergencia prematura.

REGULACIÓN DEL NÚMERO DE COPIAS

Mas adelante en la corrida, puede haber todavía diversidad significativa en la población; sin embargo la adaptación promedio de la población estará cercana a la mejor adaptación de la población.

En esta situación los miembros promedio y los mejores miembros obtienen aproximadamente el mismo número de copias en las futuras generaciones.

CODIFICACIÓN

Hay dos principios básicos para elegir la codificación en AG.

- Principio de bloques con sentido
- Principio de mínimo alfabeto

CODIFICACIÓN

Principio de bloques con sentido

El usuario debería seleccionar una codificación tal que esquemas cortos y de bajo orden sean relevantes para el problema específico y relativamente no relacionados con otros esquemas en otras posiciones fijas.

Este procedimiento es raramente práctico. El diseño de codificación para bloques con sentido es un arte.

CODIFICACION (CONT.)

Principio de mínimo alfabeto

Se debería seleccionar el alfabeto más pequeño que permita una expresión natural del problema.

Ej. los enteros $[0,31]$ se representan con 5 bits en binario (baja cardinalidad) o con 32 representaciones donde no se pueden observar similitudes relacionadas con la adaptación.

PROXIMA SEMANA

- Nuevos operadores
- La potencia de los Algoritmos Genéticos